Building Your Algorithm with VS Code and AI

RSM434H1F | Financial Trading Strategies | Fall 2026 | Setup guide

This guide takes you from a fresh laptop to running the course's base template algorithm on the RIT practice server, with an AI coding assistant ready to help you understand and improve it. You will set up **Python**, which runs your algorithm; **VS Code**, the editor where you write and run it; the **base template** from Quercus; and an **AI coding assistant** inside VS Code. It follows the setup demonstrated in class. Allow about 45 minutes the first time.

Before You Start: Windows or Mac?

Every step works on both. The base templates come in two versions, and the difference matters:

	DMA version	**REST (Client) version**
Connects to	The RIT server, directly	The RIT Client on your computer
RIT Client must be running	No	Yes
Works on	Windows and Mac	Windows only
Signs in with	Your trader ID and password	The API key shown in the RIT Client

Use the DMA version. This guide assumes it, and on a Mac it is the only option, because the RIT Client is a Windows program. The REST version is there for Windows users who prefer to trade alongside the RIT Client.

Step 1 — Install Python

If you already have Python 3.10 or later, skip to Step 2.

On Windows (10 or 11):

1. Go to https://www.python.org/downloads/ and click **Download Python install manager**. Open the downloaded file and choose **Install**. (The same app is in the Microsoft Store, as **Python Install Manager**.)
2. Open a new **PowerShell** window from the Start menu and run these two commands. The first time, the install manager may ask a few setup questions: the defaults are fine, and you can skip anything that needs administrator rights.

```
py install 3.14
python --version
```

3. The second command should print something like `Python 3.14.8`.

On a Mac:

1. Go to https://www.python.org/downloads/, download the macOS installer, and open it to install Python.
2. In Finder, open **Applications**, then the **Python 3.x** folder, and double-click **Install Certificates.command**.
3. Open **Terminal** and run `python3 --version`. On a Mac the command is `python3`, not `python`.

Step 2 — Install VS Code

1. Download VS Code from https://code.visualstudio.com/ and install it.
    - **Windows:** use the **User Installer**, which needs no administrator rights. On *Select Additional Tasks*, keep **Add to PATH** ticked, and tick both **Add "Open with Code" action** boxes, so that you can right-click a folder to open it in VS Code.
    - **Mac:** open the downloaded `.dmg` file and drag **Visual Studio Code** into your **Applications** folder, then open it from there.
2. Install the **Python** extension: open the Extensions view (**Ctrl+Shift+X** on Windows, **Cmd+Shift+X** on Mac), search for **Python**, and install the one published by **Microsoft**.

Step 3 — Set Up Your Project Folder

1. Create a folder for your algorithms, such as `RSM434-algos`. If your Documents folder syncs to OneDrive or iCloud, put it somewhere that does not sync, such as your user folder: syncing a Python environment causes problems.
2. From Quercus, download into that folder:
    - **RSM434 ALGO1 Base Template - DMA.py** and **RSM434 ALGO2e Primer - DMA.py** — the templates.
    - The **case brief** for each case. Your AI assistant can read it from the folder, which makes its suggestions far more accurate.
3. In VS Code, choose **File > Open Folder…** and open the folder. If VS Code asks whether you trust the authors of the files, choose **Yes**.

Always open the **folder**, not a single file. Your AI assistant works on the folder that is open in VS Code.

Step 4 — Choose an AI Coding Assistant

Any of the four below works with this guide. If you do not already pay for one, start with a free option.

Assistant	How to install it in VS Code	Cost (USD)
GitHub Copilot	Built in. Click the Copilot icon in the status bar, choose **Use AI Features**, and sign in with a GitHub account	**Free plan**, with monthly limits. Verified students can also get the free **Copilot Student** plan, with higher limits
Google Antigravity	Install the extension **Google Antigravity**, published by **Google** — beware of look-alikes	**Free plan**, with weekly limits. Needs a personal Google account
OpenAI Codex	Install the extension **Codex – OpenAI's coding agent**, published by **OpenAI**	Included with **ChatGPT Plus** ($20 a month) and higher
Claude Code	Install the extension **Claude Code**, published by **Anthropic**	Needs **Claude Pro** ($20 a month) or higher — not the free plan

Gemini users: Google ended free use of Gemini Code Assist in June 2026; Antigravity, which runs Google's Gemini models, replaces it. Plans and limits change often, so check the provider's site before paying for anything.

Copilot Student. Apply at https://github.com/settings/education/benefits with proof that you are enrolled, such as a timetable or enrollment letter. Approval can take several days, so start on Copilot Free in the meantime.

To install an extension, open the Extensions view (**Ctrl+Shift+X**, or **Cmd+Shift+X** on a Mac), search for its name, and check the publisher before you click **Install**. Then open the assistant from its icon in the bar down the left of the window (Copilot's chat opens from the status bar), and sign in.

Whichever you choose, use its **agent mode**, in which the assistant can read the files in your folder, edit them and run commands in VS Code's terminal. Keep it set to **ask before it runs a command**: Copilot and Claude Code ask by default; in Codex, choose **Ask for approval**; in Antigravity, set terminal commands to **Request Review**. Plain chat works too, but then you copy the code and run the commands yourself.

Step 5 — Let the Assistant Set Up Your Environment

Your algorithm needs one package, `requests`, which sends messages to the RIT server. The tidy way to install it is in a **virtual environment**: a private copy of Python inside your project folder, so that packages for this course cannot clash with anything else on your computer.

Open your assistant's chat in agent mode, paste the prompt below, and change `[Windows / macOS]` to your system. Read each command the assistant proposes before you approve it.

Prompt 1 — set up the environment

```
I'm a student in a university course on algorithmic trading. This folder
contains Python trading algorithms for the Rotman Interactive Trader (RIT)
market simulator. I use VS Code on [Windows / macOS].

Please set up this folder so I can run the .py files in it:
1. Check that Python 3 is installed and runs in VS Code's terminal. If it
   is missing, stop and tell me how to install it -- don't install it.
2. Create a virtual environment named .venv in this folder.
3. Install into .venv the packages the .py files import (I expect only
   "requests").
4. Make VS Code use .venv as this folder's Python interpreter, or tell me
   exactly how to select it.
5. Confirm that "requests" imports correctly using the .venv Python.
6. Download the RIT DMA API documentation from this address and save it
   in this folder as swagger.yaml -- I'll ask you to use it later:
   https://rit.306w.ca/RIT-DMA-API/1.0.5/swagger.yaml

Rules:
- Explain each command in one sentence before running it.
- Do not change any .py file.
- Do not run the trading algorithms: they connect to a live server.
- Never ask me for my trader ID or password.
- When you finish, list what you did and anything I still need to do.
```

When it is done, the folder contains a `.venv` folder and a `swagger.yaml` file, and the Python version shown in VS Code's status bar (bottom right) mentions `.venv`.

Prefer to do it yourself? Open VS Code's terminal (**Terminal > New Terminal**) and run these three commands, one line at a time.

On Windows:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install requests
```

On a Mac:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install requests
```

Then press **Ctrl+Shift+P** (**Cmd+Shift+P** on a Mac), run **Python: Select Interpreter**, and choose the one in `.venv`.

Step 6 — Run the Base Template

1. Open **RSM434 ALGO1 Base Template - DMA.py**. Under `settings`, replace `YOUR_TRADER_ID` and `YOUR_PASSWORD` with your own. **Type them into the file yourself — never into the AI chat.**
2. Run it while the ALGO1 practice case is running. If the case has not reached tick 5, or has passed tick 295, the template says so and stops: run it again once the case is under way.
3. Run the file: **Run > Run Without Debugging** (**Ctrl+F5**), or the triangular **Run Python File** button at the top right of the editor. The terminal shows `Trading from tick ...` and, when the case reaches tick 295, `Stopped at tick 295.`
4. To stop it early, click in the terminal and press **Ctrl+C** (on a Mac too).

The ALGO2e primer runs the same way, against the ALGO2e practice case. Each template already has the right server address for its case.

If something goes wrong, see **Troubleshooting** at the end of this guide.

Step 7 — Improve Your Algorithm with AI

The base templates work, but only just: the Class 2 slide *Making ALGO1 Robust* and the Class 3 slide *The In-Class Baseline — and What's Wrong With It* list what to fix. Work with the assistant in this order.

First, a word on the API documentation. Your algorithm talks to RIT through the **RIT DMA REST API**, and its documentation, at https://rit.306w.ca/RIT-DMA-API/1.0.5/, is the reference for every request the algorithm can send. Each *endpoint* is one kind of request: `GET /securities/book` reads a stock's order book, `POST /orders` sends an order, and `GET /limits` reports the case's trading limits. The documentation lists what each one takes and what it sends back. It also explains how the template signs in with your trader ID and password, and what happens when you send requests too fast.

Open the page in your browser to explore it yourself. Your AI assistant should use the plain-text version, `swagger.yaml`, which Prompt 1 saved in your folder: the web page builds itself in the browser, so an assistant that fetches it may see only a blank page. Without the documentation, assistants tend to invent endpoints and fields that RIT does not have. (Using a REST template instead? Its documentation is at https://rit.306w.ca/RIT-REST-API/1.0.5/.)

Understand the code first. You cannot improve, or defend, code you do not understand.

Prompt 2 — explain the code

```
Explain the algorithm in [file name] to me as if I'm new to Python.
Walk through the trading logic first, then each function, and explain
each request it sends to RIT using the RIT DMA API documentation in
swagger.yaml. Then list its weaknesses as a trading strategy, using the
case brief in this folder. Don't change anything.
```

Then make one change at a time. Give the assistant the context it needs — the case rules, the API documentation, your current code — and one specific task.

Prompt 3 — make one improvement

```
I'm improving my Python algorithm for the RIT [ALGO1 / ALGO2e] case, in
[file name]. It connects through the RIT DMA REST API. First read the
case brief in this folder and the API documentation in swagger.yaml, and
use only endpoints and fields that the documentation describes.

Task: [ONE specific change -- for example, "when my net position is long,
lower my bid and ask by one cent; when it is short, raise them"].

Before editing, save a copy of the current file in a "backups" folder,
numbered (v1, v2, ...), and explain your plan. Change only what the task
needs. Afterwards, explain each change, and point out anything that could
breach the position limits or send requests faster than RIT allows. Do
not run the algorithm.
```

Check its work. A second pass catches mistakes that are typical of AI-written trading code.

Prompt 4 — review for bugs

```
Review [file name] for bugs that would lose money or break the rules in
the RIT [ALGO1 / ALGO2e] case. Check in particular for: gross and net
positions mixed up; signs flipped for short positions; orders assumed to
have filled just because they were accepted; loops with no pause; and
API endpoints or fields that are not in the RIT DMA API documentation
in swagger.yaml.
List each problem and why it matters. Don't fix anything yet.
```

Test every change on the practice server before the next one, and watch your **position**, not just your P&L.

Using AI Responsibly

- **Every order your algorithm sends is yours.** Read every line you run. If you cannot explain it, do not run it — and never run untested code in a graded evaluation.
- **Cite and document.** The course encourages generative AI. Under the course outline, content produced by AI must be cited, and its use documented in an appendix to each assignment: which tools you used, how, and how you incorporated the results.
- **Keep your password private.** The assistant sends what it reads in your folder to its provider. Never type your RIT password into the chat, and replace it with `YOUR_PASSWORD` before you share a file with anyone. Some free plans also use your prompts and code to train their models unless you opt out in the assistant's settings.
- **Keep a working version.** Prompt 3 saves a numbered backup before each change. If a change makes things worse, go back to the last good backup rather than piling fixes on top.

Troubleshooting

Problem	What to do
Windows: `python` opens the Microsoft Store, or is "not recognized"	In the Start menu, open **Manage app execution aliases** and make sure the **Python (default)** aliases are on — turn them off and on again. Then open a new terminal
Mac: `python: command not found`	Use `python3`. Inside an activated `.venv`, `python` works too
Windows: activating `.venv` fails because "running scripts is disabled"	Run `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser` once in PowerShell, then try again
`ModuleNotFoundError: No module named 'requests'`	VS Code is running a different Python. Select the `.venv` interpreter (Step 5), open a new terminal, and run the file again
`could not read the case (HTTP 401)`	Your trader ID or password is wrong. (In the REST version, your API key is wrong)
A connection error, or a long wait and then a timeout	Check your internet connection, and that `BASE_URL` is still as it was in the template. If it still fails, the practice server may be down: let the teaching team know
`The case is at tick 0`	The case has not started, or has finished. Run the algorithm again once the case is under way
The ALGO2e primer ends at once, without trading	It trades only while the case is running. Start it again once the case is under way
The algorithm pauses now and then	Usually normal: when RIT says you are sending requests too fast, the template waits as long as RIT asks and then resends
The assistant cannot read a PDF	Copy the relevant part, such as the case's table of rules, into the chat
There is no `swagger.yaml` in your folder	Open https://rit.306w.ca/RIT-DMA-API/1.0.5/swagger.yaml in your browser and save the file into your project folder
The assistant uses an API endpoint or field that does not exist	Tell it to check against `swagger.yaml` — and see Prompt 4